第 2-4 讲 • 装修队：React 与界面

> **模块 2 • 认清记账簿的骨架** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成 2-1 ~ 2-3

本节目标

读完并做完这一讲，你能：

1. 说清 **React 的核心思想**："界面是数据的投影"，以及它和"用代码手工改界面"的区别。
2. 看懂一个**组件（Component）**：它是什么、长什么样、为什么能重复使用。
3. 理解**状态（State）**：界面上"会变的东西"存在哪、变了之后发生什么。
4. 说出记账簿界面部分的**真实规模**（19 个页面 + 4 个通用组件 + 2 个状态仓库）。
5. **亲手改一次界面**（改文字、改颜色），并在浏览器/窗口里看到变化。

一、React 的核心思想：界面是数据的投影

1.1 现实比喻：公告栏 vs 手写海报

老做法（手工改界面）：

想象你经营一家店，门口贴着"今日特价：苹果"。苹果卖完了，你得**自己**：

1. 找到那张纸
2. 把它撕下来
3. 写上"今日特价：香蕉"
4. 重新贴上去

如果店里贴了 20 张纸（价格、库存、营业时间……），每次数据变化你都要知道"哪几张纸要改"。**漏一张，顾客就看到过时信息。**

React 的做法（界面是数据的投影）：

你在墙上装一块**电子屏**，屏幕上的内容由一份数据决定：

```
数据：{ 特价商品："苹果"，库存：3 }
    ↓ 投影
屏幕显示：今日特价 苹果（剩 3 件）
```

苹果卖完了，你只改**数据**（`特价商品 = "香蕉"`），**屏幕自动更新**——你根本不用管"哪一行要改"。

这就是 React 的核心：

> **你不直接操作界面，你只描述"界面应该长什么样"（基于数据）。数据一变，React 负责把界面更新成对应的样子。**

为什么这样更好：

老做法	React
你要记住"哪些地方显示了金额"，逐个更新	只改数据，所有显示金额的地方自动更新
漏改一处就会出现"界面和数据不一致"	不可能不一致——界面就是数据算出来的
界面上百个元素时，逻辑会失控	规模变大也不会失控

1.2 一个真实的例子

记账簿里显示金额的组件（`MoneyText`），它的逻辑是：

```
你给它一个金额（分）→ 它按设置决定显示什么
  · 币种是人民币 → 显示 ¥12.00
  · 开启了美元显示 → 同时显示 $1.67
```

它不关心"这笔钱是谁花的""该不该显示正负号"（那由传进来的参数决定）。**它只负责一件事：把数字变成好看的文字。**

这种"只干一件事"的组件，可以到处复用：记录列表用它、汇总页用它、预算页也用它。

改一次，全站生效。 这就是"组件化"的价值。

二、三个必须理解的概念

2.1 组件（Component）

组件 = 一块可复用的界面积木。

在代码里，组件就是一个**返回界面结构的函数**：

```
export function MoneyText({ cents }: { cents: number }): JSX.Element {
  return <span>¥{(cents / 100).toFixed(2)}</span>
}
```

几个要点：

概念	说明
函数名**首字母大写**	这是约定，React 靠它区分"组件"和"普通 HTML 标签"
括号里的参数	叫 **props（属性）**，是"外面传进来的信息"
`return` 的东西	看起来像 HTML 的写法（叫 JSX），**实际是描述界面结构**
文件扩展名 `.tsx`	带 JSX 的 TypeScript 文件

记账簿的真实规模：

类型	数量	例子
页面组件	**19 个**	记录页、日历页、汇总页、预算页……
通用组件	**4 个**	`MoneyText`、`Icon`、`Toaster`、`ui.tsx`（按钮/卡片/弹窗合集）

19 个页面，每个页面就是一块大积木，由更小的积木拼起来。

2.2 状态（State）

状态 = 界面上"会变的数据"。

比如"当前选中的月份"、"弹窗开着还是关着"、"列表里有多少条记录"——这些都会变，React 把它们叫**状态**。

关键规律：

```
状态变了  →  React 自动重新计算界面  →  你看到新内容
```

你不需要写代码去"改界面"，只需要改状态。

一个真实例子——记账簿里切换月份：

```
const [month, setMonth] = useState('2026-09')   // 状态：当前月份


// 用户点"上一月"时：
setMonth('2026-08')      // 只改状态
// → 界面上的月份、账目列表、合计数字，全部自动更新
```

注意 `useState` 这个写法（读作 "use state"）：它声明"我要一个会变的值"，并给你两个东西——**当前值**和**修改它的函数**。

2.3 状态放在哪：两种选择

这是本项目一个值得学的设计。界面上会变的数据分两类：

类型	放在哪	例子
只影响这一块	**组件内部的 `useState`**	某个弹窗开着没、输入框里的文字
很多地方都要用	**全局状态仓库（store）**	当前设置、账户列表、分类列表

记账簿有两个全局仓库（真实文件）：

```
src\renderer\src\stores\
├─ app.store.ts      ← 应用级状态：设置、账户、分类
└─ toast.store.ts    ← 提示消息（右下角浮出来的那些）
```

为什么设置要放全局：因为"用户的币种设置"会影响**每一个显示金额的地方**。如果放在某个组件内部，别的组件就拿不到，只能一层层传下去——那会非常麻烦。

判断标准（记住这个）：

> **只有这个组件用的 → 放组件里；好几个地方都要用的 → 放全局仓库。**

三、在项目里看真实文件

3.1 界面的入口

```
src\renderer\src\main.tsx
```

打开它，你会看到：

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <HashRouter>
    <App />
  </HashRouter>
)
```

逐行翻译：

代码	意思
`document.getElementById('root')`	找到 HTML 里那个空的挂载点（`<div id="root">`）
`createRoot(...)`	告诉 React："从这里开始接管界面"
`.render(<App />)`	把整个应用渲染进去
`<HashRouter>`	路由：根据网址决定显示哪个页面

所以整个界面的启动只有一个动作：把 `<App />` 渲染到 `#root` 里。 剩下的一切都是这个组件往下展开的。

3.2 页面的组织

```
src\renderer\src\
├─ App.tsx           ← 路由表（哪个网址显示哪个页面）
├─ layouts\
│   └─ AppLayout.tsx ← 外壳（侧栏 + 顶部栏），所有页面共用
├─ features\
│   ├─ records\      ← 记录页
│   ├─ calendar\     ← 日历页
│   ├─ stats\        ← 汇总页
│   └─ ...（共 19 个页面组件）
├─ components\       ← 通用积木（4 个）
└─ stores\           ← 全局状态（2 个）
```

`layouts/AppLayout.tsx` **很值得一看**：它是**所有页面共用的外壳**（侧栏导航 + 主题按钮）。里面有个导航列表：

```
const NAV = [
  { to: '/records', label: '记录', icon: 'records' },
  { to: '/billshot', label: '截图记账', icon: 'billshot' },
  ...
]
```

这个列表生成整个侧栏。你加一项，侧栏就多一项——**这就是"界面是数据的投影"的直接体现**：导航项不是手写死的 HTML，而是**由一份数据（`NAV` 数组）算出来的。**

你在第 0-2 讲改的就是这里（把 `'记录'` 改成别的文字），当时窗口自动刷新了——**因为你改了数据，界面就跟着变。**

3.3 一个完整的组件

打开 `src\renderer\src\components\MoneyText.tsx`，你能看到一个典型组件的样子：

```
export function MoneyText({ cents, type, signed = false, countUp = false }) {
  const settings = useAppStore((state) => state.settings)   // 从全局仓库读设置
  const shown = useCountUp(cents, { enabled: countUp })      // 数字滚动动画
  return <span>...</span>                                     // 返回界面结构
}
```

三件事同时发生：

1. **接收 props**（`cents`、`type`……）—— 外面传进来的
2. **读全局状态**（`settings`）—— 币种、是否显示美元
3. **返回界面** —— 根据前两者算出来

这就是一个 React 组件的完整生命周期。

四、动手任务

步骤 1：启动开发模式

```
cd "C:\Users\34123\Desktop\记账簿"
npm run dev
```

等窗口出现（如果已经开着，跳过）。

步骤 2：改侧栏文字（复习第 0-2 讲）

打开 `src\renderer\src\layouts\AppLayout.tsx`，找到 `NAV` 数组，把：

```
{ to: '/records', label: '记录', icon: 'records' },
```

改成：

```
{ to: '/records', label: '📒 我的账本', icon: 'records' },
```

存盘 → 看窗口：侧栏那一项变成了「📒 我的账本」。

这一步的意义：你**改了数据**（`NAV` 数组的一项），界面自动跟着变。**这就是"界面是数据的投影"。**

步骤 3：再改一项，验证"投影"的威力

还是刚才那个数组，把「日历」那项的 `label` 也改掉：

```
{ to: '/calendar', label: '看日子', icon: 'calendar' },
```

存盘：侧栏第二项、第三项都变了，**而你只改了一个数组里的两行。**

关键领悟：你没有去修改任何 HTML、没有去找"侧栏第二项那个元素"——**你只改了描述界面的数据。**

步骤 4：改样式，看颜色变化

打开 `src\renderer\src\styles\index.css`，找到这一行（浅色主题的主色）：

```
--c-accent: #2e9e93;
```

把它改成：

```
--c-accent: #d97706;
```

存盘 → 观察窗口：

整个软件的青色全部变成了橙色 —— 按钮、选中项、图标、进度条……

为什么：这个项目**没有把颜色写死在各个组件里**，而是集中定义在几个"变量"上，各处引用变量。

这是"可维护性"的经典做法：想换主题，改一个值就够了，不用去 50 个文件里找 `#2e9e93`。

改完记得改回去（或者留着，做完实验再改回来）。

步骤 5：观察"状态"的存在

1. 在记录页，点右上角切换月份（`‹` `›`）。
2. **打开 VS Code 的搜索**，在 `RecordsPage.tsx` 里找 `useState`：

```
Select-String -Path src\renderer\src\features\records\RecordsPage.tsx -Pattern "useState"
```

预期看到类似 `const [month, setMonth] = useState(...)`、`const [formOpen, setFormOpen] = useState(false)` 等。

每一个 `useState` 就是一个"会变的东西"：

状态	管什么
`month`	当前看的月份
`formOpen`	新增记录的弹窗开着没
`editing`	正在编辑哪一条

点一下"上一月"，实际上是 `setMonth(...)` 被调用了，然后界面重新算了一遍。

卡住了怎么办

现象	原因	怎么办
改完没反应	没存盘 / dev 没在跑	确认存盘；确认终端里 `npm run dev` 还在运行
窗口报错、一片红	改坏了语法（比如少了个逗号）	看终端报错的行号，改回去
改了 CSS 颜色没变	可能改的是深色主题那段	找 `html[data-theme='dark']` 下面那段才是深色
改完想还原	不记得原值	青色是 `#2e9e93`，深色主题是 `#48b4a7`

五、自测题

1. "界面是数据的投影"是什么意思？举例说明它比"手工改界面"好在哪。

2. 组件是什么？为什么组件名首字母要大写？

3. `useState` 给你哪两个东西？状态变化后会发生什么？

4. 界面上会变的数据，什么时候该放在组件内部，什么时候该放全局仓库？

5. 你给侧栏 `NAV` 数组加了一项，为什么侧栏就多了一个按钮?

答案与解析

1. 意思是：**你不直接操作界面，只描述"界面按数据应该长什么样"**，数据变了界面自动更新。 → 好处：改动只需改数据，**所有引用它的地方自动同步**，不可能出现"界面和数据不一致"。手工改界面则要记住"哪几处显示了它"，漏一处就出错。

2. 组件是**一块可复用的界面积木**，本质是一个返回界面结构的函数。 → 首字母大写是 React 的约定：**用它区分"自定义组件"和"普通 HTML 标签"**（`<div>` 是标签，`<MoneyText>` 是组件）。

3. 给你**当前值**和**修改它的函数**（如 `month` 和 `setMonth`）。 → 调用修改函数后，React **自动重新计算界面**并更新显示，你不用写"怎么改界面"的代码。

4. 只有这个组件自己用的（某个弹窗开没开、输入框里的字）→ 放组件内部；**好几个地方都要用的**（设置、账户列表）→ 放全局仓库（store）。 → 判断标准：**会不会有别的组件也需要它。**

5. 因为侧栏是**由 `NAV` 数组"投影"出来的**——代码里用 `NAV.map(...)` 遍历数组生成按钮，而不是手写死的 HTML。 → 加一项数据，就多一个按钮。**这正是"界面是数据的投影"的直接证据。**

六、术语表

术语	一句话解释
React	一个界面库，核心思想是"界面是数据的投影"
组件 / Component	可复用的界面积木，本质是返回界面结构的函数
props / 属性	从外面传给组件的信息
JSX	在 JavaScript 里写界面结构的语法，长得很像 HTML
`.tsx`	含 JSX 的 TypeScript 文件
状态 / State	界面上会变的数据
`useState`	声明一个会变的值，返回"当前值 + 修改函数"
仓库 / Store	全局状态容器，多个组件共享的数据放这里
渲染 / Render	React 根据数据算出界面结构并显示出来的过程
重渲染 / Re-render	数据变化后，React 重新计算界面
路由 / Router	根据网址决定显示哪个页面
挂载点 / Root	HTML 里那个空的 `#root`，React 从这里接管界面
可维护性	改动一处就能影响全站、且不容易出错的性质
CSS 变量	集中定义的颜色等值，改一处全站生效

七、本讲与项目的对应

- 你改动的文件：`AppLayout.tsx`（`NAV` 数组）、`styles\index.css`（主色变量）
- **本讲可以完全还原**（改回原值即可），**不影响你的记账数据**
- 上一讲：2-3 冷库：数据库与迁移 ｜ **下一讲进入模块 3：3-1 读代码的地图与心法**

八、模块 2 小结

到这里，记账簿的**全部骨架**你已经认识了：

讲	你学到的
2-1	**主进程 / 渲染进程**：后厨与服务员，为什么必须分开
2-2	**preload / IPC**：传菜口与点菜单，一条请求的四个环节
2-3	**数据库 / 迁移**：冷库、整数存分、软删除、版本升级
2-4	**React / 组件 / 状态**：界面是数据的投影

这四块拼起来，就是一个完整的记账软件。 剩下的所有代码，都是这四块的具体实现。

模块 3 开始，你要学会"自己读"——不再是我告诉你每个文件干什么，而是给你一套方法，让你**拿到任何一段代码都能自己搞明白。**

讲义版本：v1 ｜ 对应项目版本：0.1.0